🐳 Docker — The Complete Notes

> A deep, visual, and practical guide to Docker — from "what is a container" all the way to multi-stage builds, networking, volumes, Compose, and production best practices.

📚 Table of Contents

1. Why Docker Exists

> "It works on my machine." — Every developer, ever.

Docker solves the classic problem of **environment inconsistency**. Before Docker, deploying an app meant:

- Installing the right OS
- Installing the right runtime (Node, Python, Java...)
- Installing the right system libraries
- Praying nothing conflicts with other apps on the server

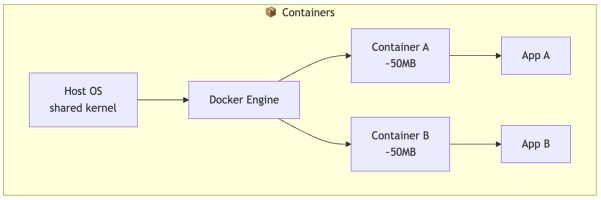
Docker packages your app + all its dependencies into a single portable unit called a container. That unit runs *identically* on your laptop, your CI server, and production.

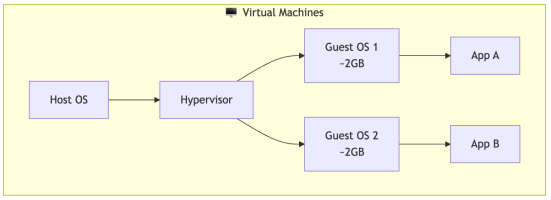
Key benefits

Benefit	What it means
Portability	Build once, run anywhere (Linux, Mac, Windows, cloud)
Isolation	Each container has its own filesystem, network, processes
Lightweight	Shares the host kernel — starts in milliseconds
Reproducible	Dockerfile = recipe = same result every time
Scalable	Spin up 1 or 1000 identical containers

2. Virtual Machines vs Containers

This is the **most important mental model** to internalize.

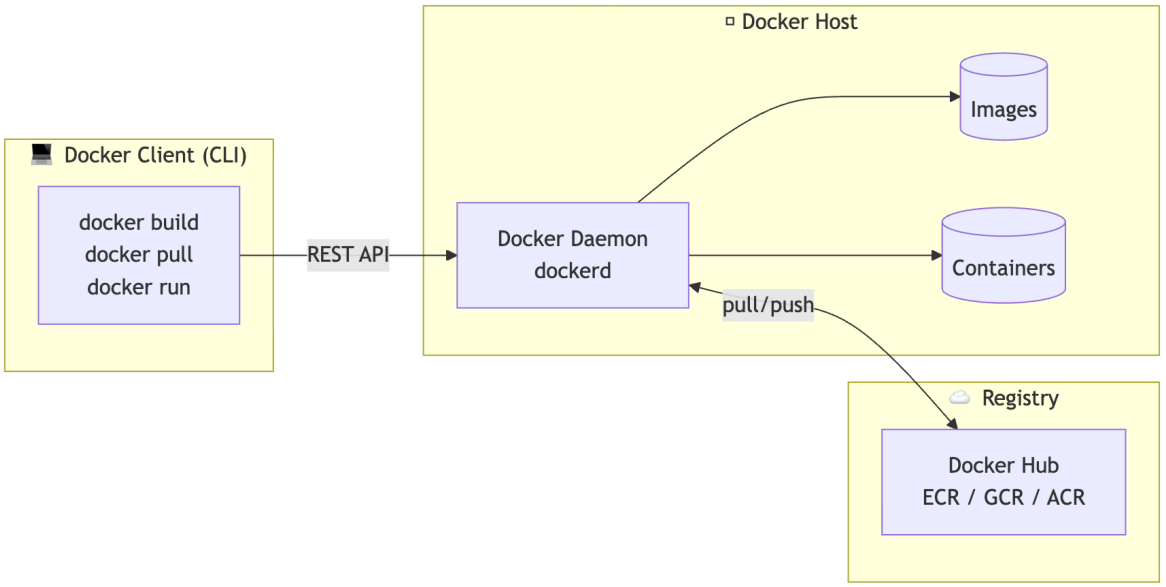




Aspect	Virtual Machine	Container
Boot time	Minutes	Milliseconds
Size	GBs	MBs
OS	Each VM has its own	Shares host kernel
Isolation	Hardware-level (strong)	Process-level (weaker but enough)
Performance	Overhead from guest OS	Near-native

3. Docker Architecture

Docker uses a **client–server architecture**.



The three pieces

1. **Docker Client** — what you type commands into (`docker run ...`).
2. **Docker Daemon (`dockerd`)** — the background service that actually does the work: builds images, runs containers, manages networks/volumes.
3. **Registry** — remote storage for images (e.g., Docker Hub).

4. Core Concepts: Images, Containers, Registries

🧱 Image

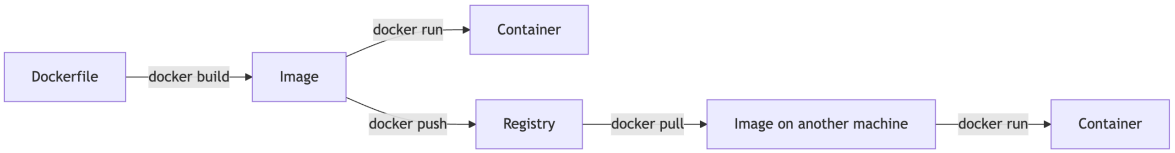
A **read-only template** that contains your app + its environment. Think of it like a class.

📦 Container

A **running instance** of an image. Think of it like an object instantiated from a class.

🏬 Registry

A **storage and distribution system** for images.

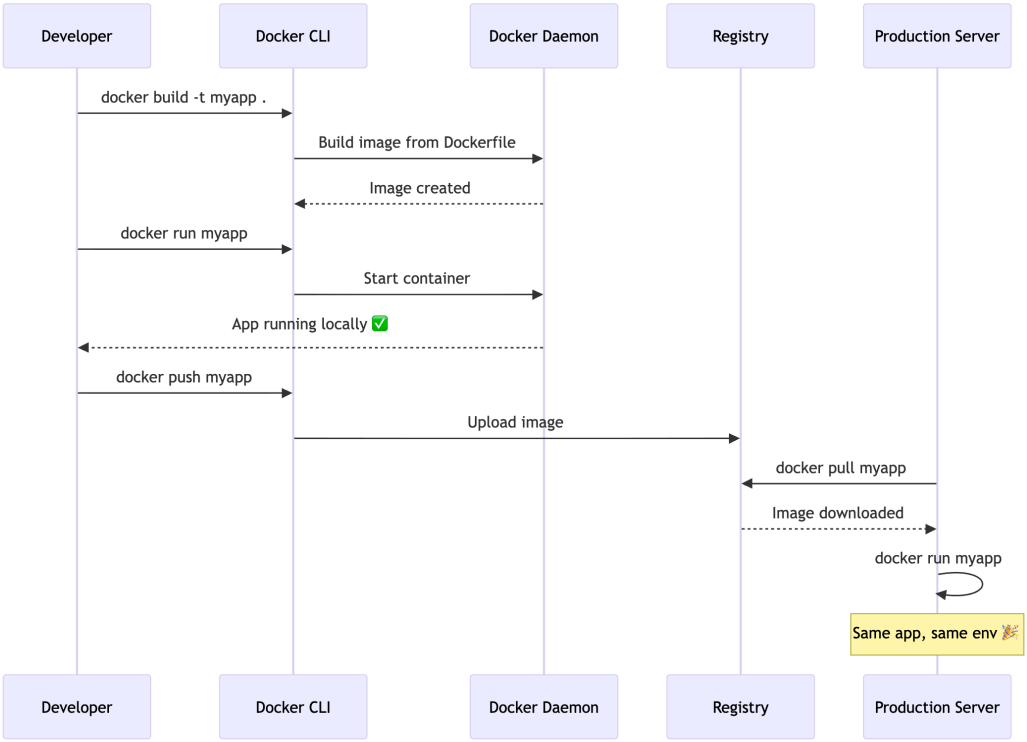


Analogy

Docker	Cooking
Dockerfile	Recipe
Image	Prepared meal kit
Container	The cooked dish on your plate
Registry	Grocery store

---

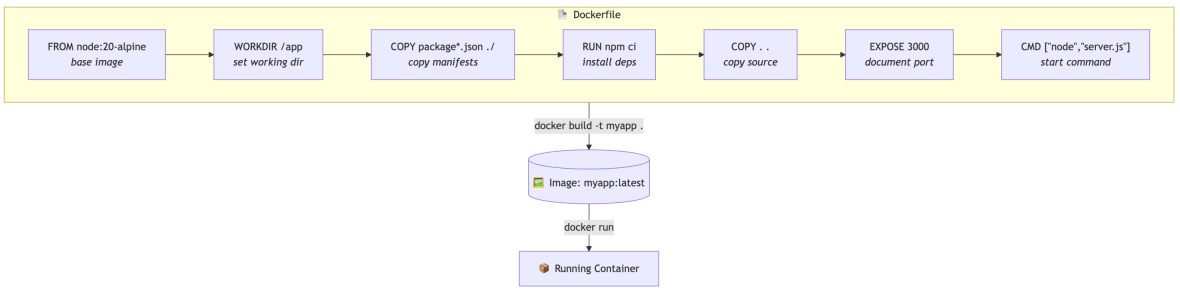
5. The Docker Workflow



---

6. Dockerfile Deep Dive

A **Dockerfile** is a text file with instructions to build an image.



Minimal example (Node.js app)

```
# 1. Base image
FROM node:20-alpine

# 2. Set working directory inside the container
WORKDIR /app

# 3. Copy dependency manifests first (better caching)
COPY package*.json ./

# 4. Install dependencies
RUN npm ci --only=production

# 5. Copy the rest of the source
COPY . .

# 6. Document the port the app listens on
EXPOSE 3000

# 7. Default command when the container starts
CMD ["node", "server.js"]
```

Instruction reference

Instruction	Purpose
`FROM`	Base image to start from
`WORKDIR`	Sets the working directory for subsequent commands
`COPY` / `ADD`	Copy files from host into image
`RUN`	Execute a command at **build time** (creates a new layer)
`ENV`	Set environment variables
`ARG`	Build-time variables
`EXPOSE`	Document which port the container listens on
`CMD`	Default command at **runtime** (one per Dockerfile)
`ENTRYPOINT`	Configures the container to run as an executable
`USER`	Run as a non-root user
`HEALTHCHECK`	Tells Docker how to test if the container is healthy

`CMD` vs `ENTRYPOINT`

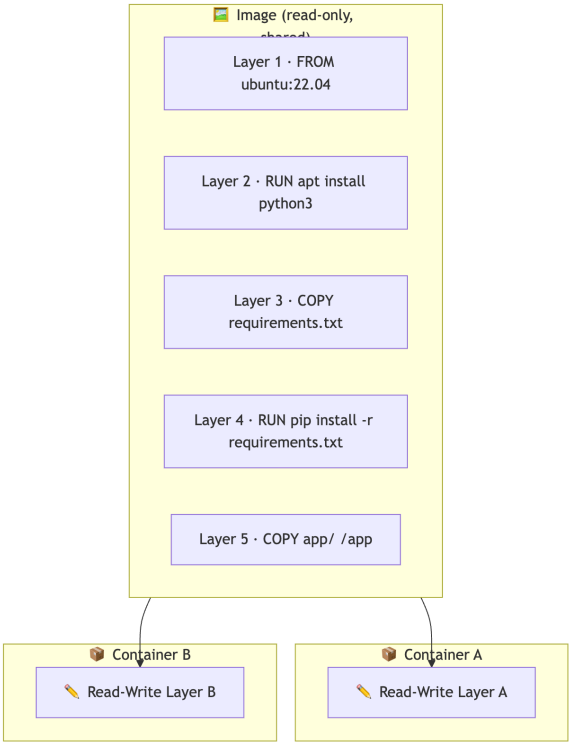
```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Runs as `python app.py`. If user runs `docker run myimg other.py`, it becomes `python other.py` — `CMD` is overridable, `ENTRYPOINT` is not.

---

7. Image Layers & Caching

Every instruction in a Dockerfile creates a **layer**. Layers are **cached** and **stacked**.



Each container gets its own thin writable layer on top of the same shared, read-only image layers.

Why order matters

If you change one line in your source code, Docker invalidates that layer **and every layer after it**. So put things that change *least* at the top:

```
# ✅ GOOD — deps cached unless package.json changes
COPY package*.json ./
RUN npm ci
COPY . .

# ❌ BAD — every source change reinstalls deps
COPY . .
RUN npm ci
```

`.dockerignore`

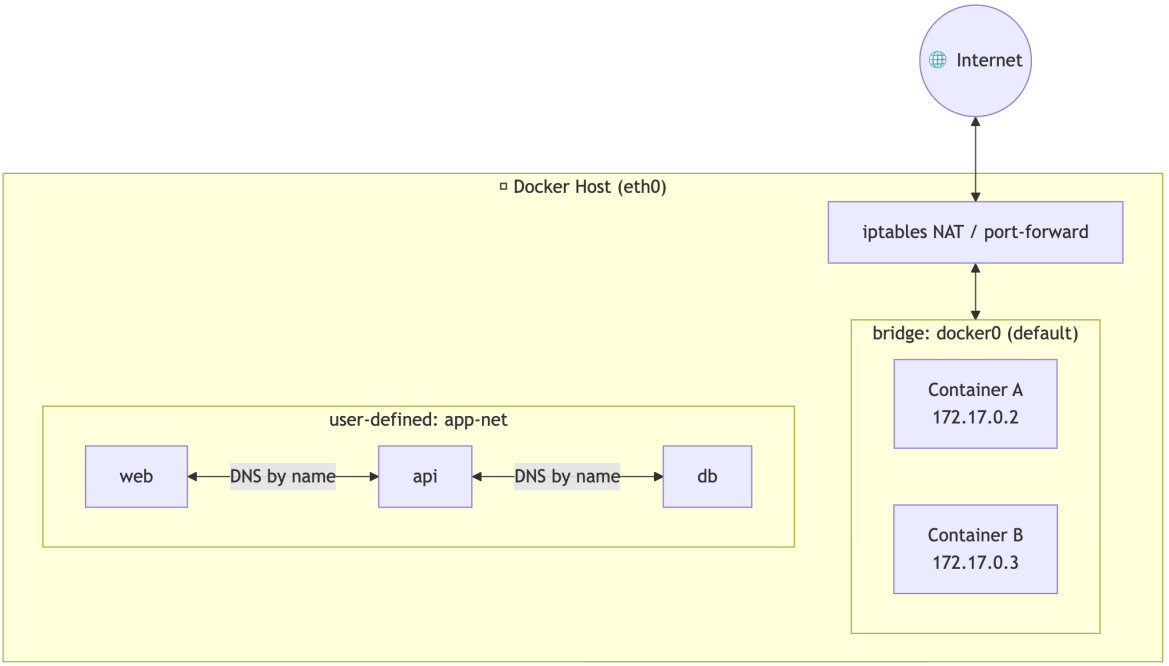
Like `.gitignore`. Keeps junk out of the build context:

```
node_modules
.git
.env
*.log
dist
```

---

8. Docker Networking

Containers need to talk to each other and to the outside world.



Network drivers

Driver	Use case
`bridge`	**Default.** Containers on the same host can talk to each other
`host`	Container shares the host's network stack (no isolation)
`none`	No networking
`overlay`	Multi-host networking (Docker Swarm / Kubernetes)
`macvlan`	Container gets its own MAC address on the network

Port mapping

```
docker run -p 8080:3000 myapp
#          host:container
```

Host's port 8080 → Container's port 3000.

DNS between containers

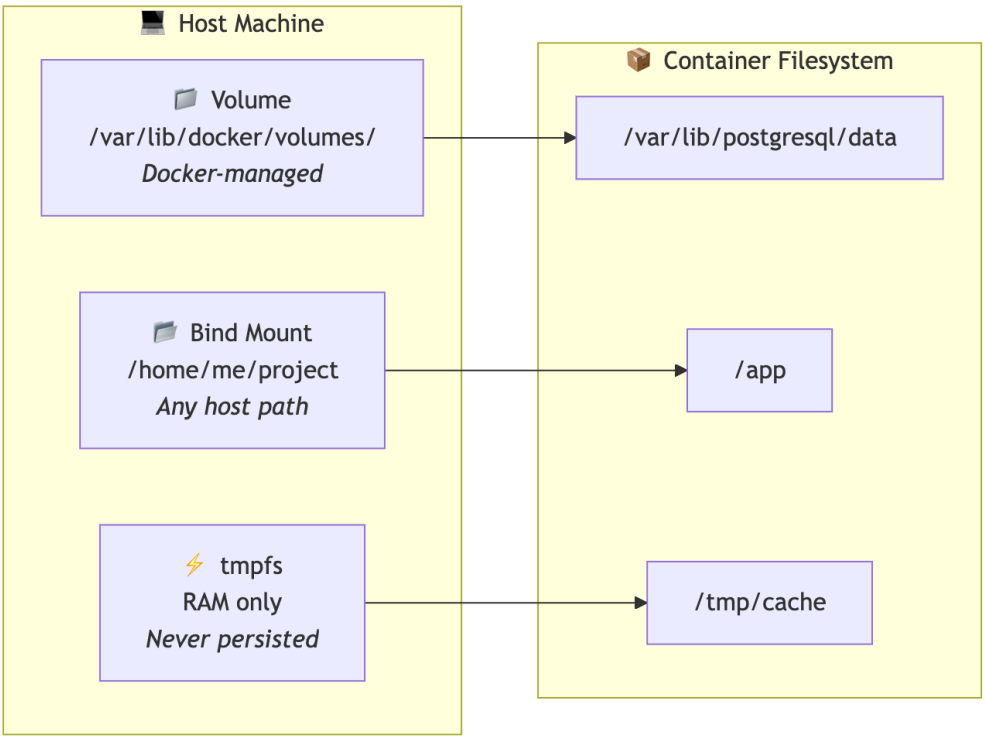
On a **user-defined bridge network**, containers can reach each other **by container name**:

```
docker network create app-net
docker run -d --name db   --network app-net postgres
docker run -d --name web  --network app-net myapp
# Inside 'web', the DB is reachable at host "db"
```

---

9. Docker Volumes & Storage

Containers are **ephemeral** — when they die, their data dies with them. Volumes solve this.



Comparison

Type	Stored where	Best for
Volume	Docker-managed dir	Production data (DBs)
Bind mount	Anywhere on host	Dev (live code reload)
tmpfs	Host RAM	Secrets, temp data

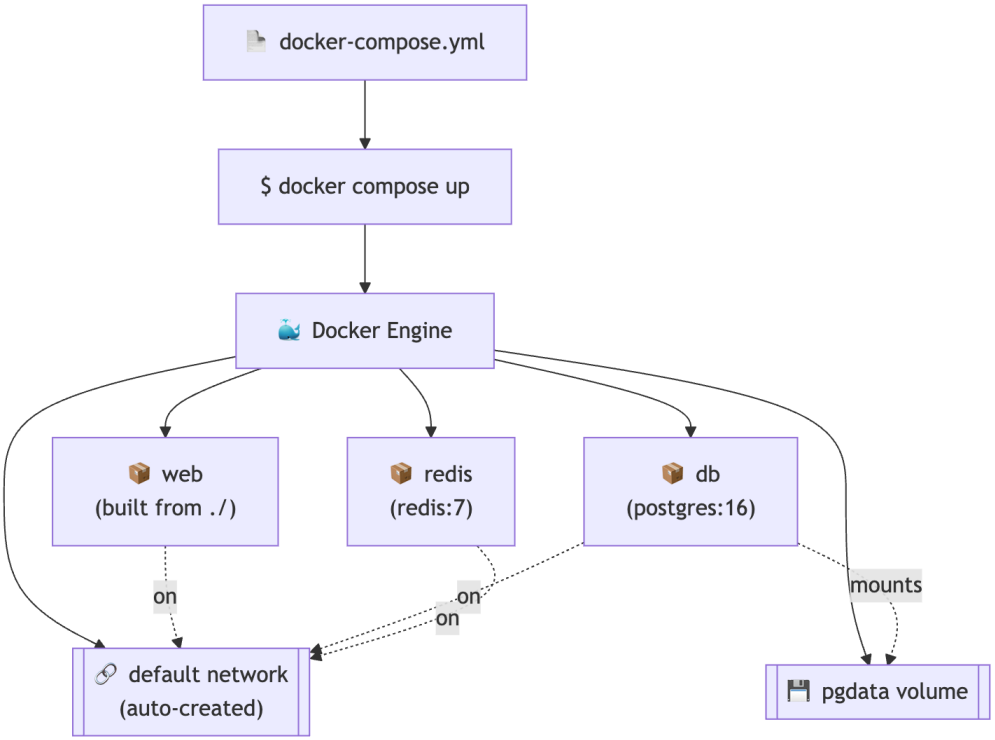
Examples

```
# Named volume (recommended for prod)
docker run -v pgdata:/var/lib/postgresql/data postgres

# Bind mount (dev)
docker run -v $(pwd):/app node:20

# tmpfs
docker run --tmpfs /tmp myapp
```

---

10. Docker Compose

When your app needs **multiple services** (web + DB + cache), running `docker run` ten times is painful. Enter **Compose**.



`docker-compose.yml` example

```yaml
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/mydb
    depends_on:
      - db
      - redis

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

volumes:
  pgdata:
```

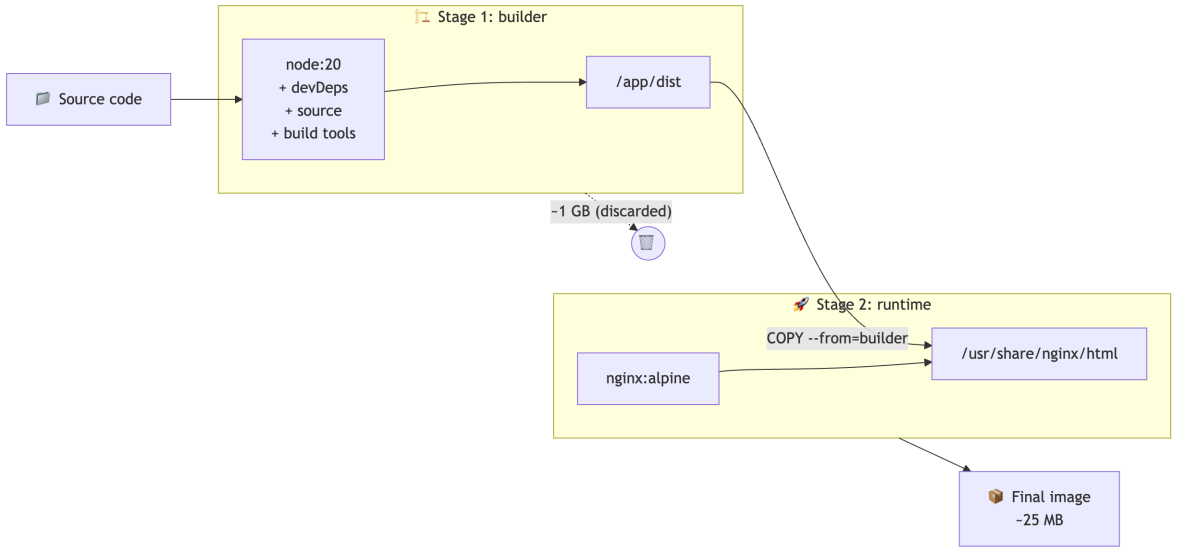
Key commands

```
docker compose up -d        # start everything in background
docker compose logs -f web  # tail logs of one service
docker compose ps           # list services
docker compose down         # stop & remove
docker compose down -v      # also remove volumes
```

---

11. Multi-Stage Builds

Build in one image, ship a tiny final image.




```
# ----- Stage 1: build -----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build         # produces /app/dist

# ----- Stage 2: run -----
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Result: final image contains only nginx + static files (~~25 MB) instead of Node + node_modules + source (~~1 GB+).

---

12. Docker Registry & Docker Hub

A registry stores and distributes images.

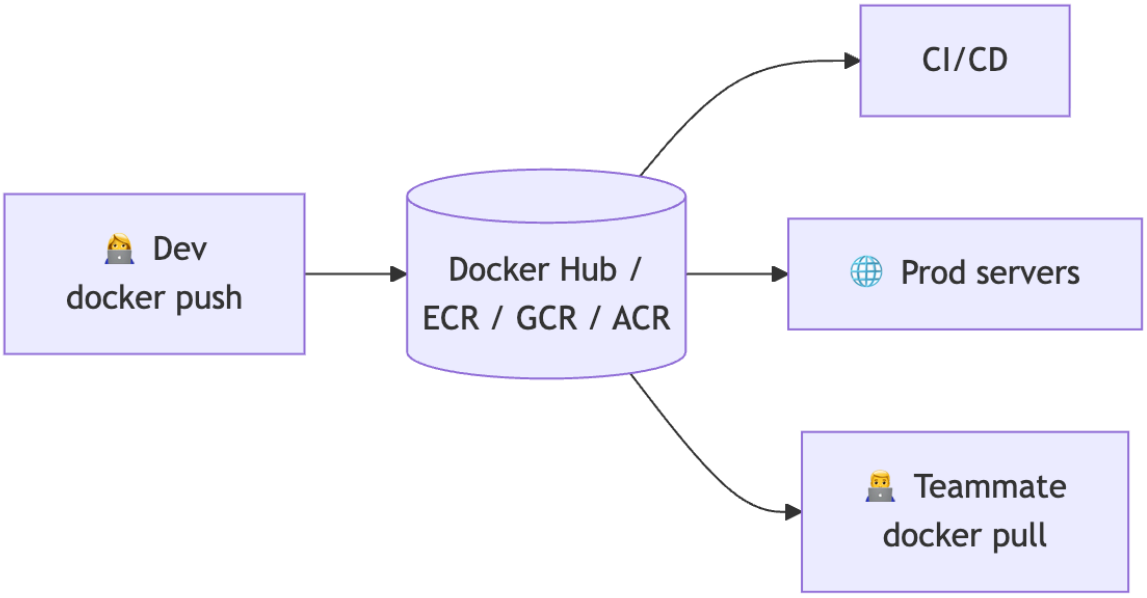


Image naming

```
registry.example.com/namespace/repo:tag
docker.io/library/nginx:1.25-alpine
```

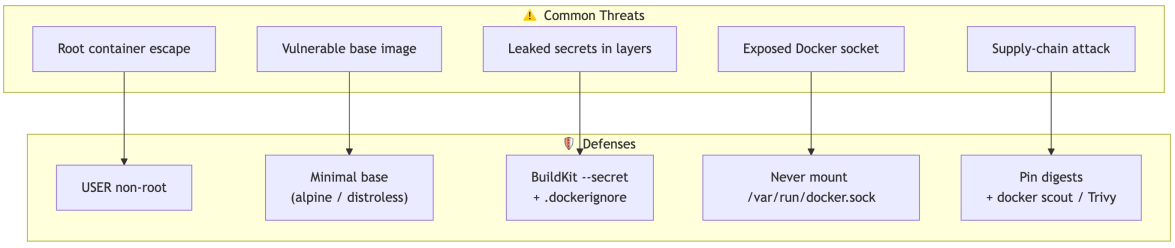
If you omit the registry, it defaults to `docker.io` (Docker Hub).

Common commands

```
docker login
docker tag myapp:latest user/myapp:1.0
docker push user/myapp:1.0
docker pull user/myapp:1.0
```

---

13. Security Best Practices



✅ Do

- **Use minimal base images** (`alpine`, `distroless`, `slim`).
- **Pin versions** — `node:20.11.1-alpine`, not `node:latest`.
- **Run as non-root**:

  ```
  RUN addgroup -S app && adduser -S app -G app
  USER app
  ```

- **Scan images** — `docker scout cves myimage`.
- **Use `.dockerignore`** to avoid leaking secrets.
- **Don't bake secrets** into images — use env vars, secrets managers, or BuildKit `--secret`.

- **Use multi-stage builds** to drop build tools from the final image.
- **Read-only filesystem** where possible:

```
docker run --read-only myapp
```

❌ **Don't**

- `chmod 777` everything.
- Run as root in production.
- Use `ADD` for remote URLs (use `RUN curl` + checksum).
- Expose the Docker socket (`/var/run/docker.sock`) unnecessarily — it's effectively root on the host.

---

14. Common Commands Cheat Sheet

Images

```
docker images                       # list
docker pull nginx:alpine            # download
docker build -t myapp:1.0 .         # build
docker rmi myapp:1.0                # remove
docker image prune -a               # remove unused
```

Containers

```
docker ps                           # running
docker ps -a                        # all
docker run -d --name web -p 80:80 nginx
docker exec -it web sh              # shell into container
docker logs -f web                  # tail logs
docker stop web && docker rm web
docker container prune              # remove stopped
```

Inspection

```
docker inspect web                  # full JSON details
docker stats                        # live CPU/mem
docker top web                      # processes inside
docker diff web                     # filesystem changes
```

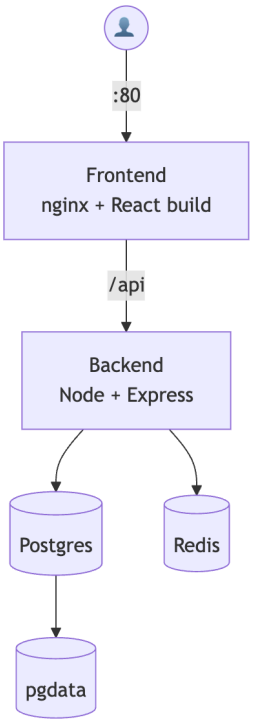
System

```
docker system df                    # disk usage
docker system prune -a --volumes    # 🧹 nuke everything unused
```

---

15. Real-World Example: Fullstack App

Let's containerize a typical app: **React frontend + Node API + Postgres.**



docker-compose.yml

```yaml
services:
  frontend:
    build: ./frontend
    ports: ["80:80"]
    depends_on: [api]

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
      REDIS_URL: redis://cache:6379
    depends_on: [db, cache]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app
    volumes: [pgdata:/var/lib/postgresql/data]

  cache:
    image: redis:7-alpine

volumes:
  pgdata:
```

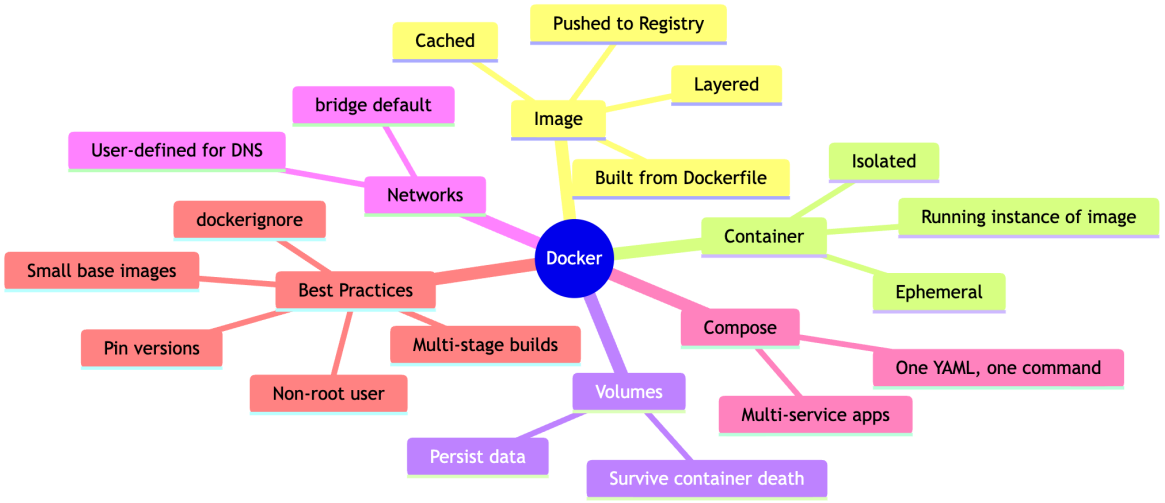
Run with one command:

```
docker compose up -d --build
```

16. Troubleshooting

Problem	Fix
`Cannot connect to the Docker daemon`	Start Docker Desktop / `sudo systemctl start docker`
Image too big	Use Alpine base + multi-stage build
Container exits immediately	`docker logs <id>` — usually a crash or wrong `CMD`
Port already in use	`docker ps` to find conflict, or change host port
Changes not showing up	Rebuild without cache: `docker build --no-cache .`
"No space left on device"	`docker system prune -a --volumes`
Slow build	Reorder Dockerfile so changing layers come last
Can't reach another container	Put them on the same user-defined network

🎯 TL;DR



📖 Further Reading

- 📘 Official Docker Docs
- 📗 Docker Curriculum (free)
- 📕 Play With Docker (in-browser lab)
- 📙 Awesome Docker (curated list)

> **Remember:** Docker is just a tool. The real skill is thinking in **immutable, reproducible, isolated units**. Once that clicks, Kubernetes, CI/CD, and modern DevOps all start making sense.